Digital Assignment 2

Name: KAILASH P
Register No: 24BRS1382
Course: Software Engineering
GitHub Repository: https://github.com/kailashp-27/EchoSphere
Figma Prototype: https://www.figma.com/make/o0oFJ9qgmdbQpxuekEYeyl/EchoSphere-Wireframe-Outline?t=tsLKVe3ktEjJpQcJ-20&fullscreen=1
Drawio:https://drive.google.com/file/d/1ni_RgGjN7F9XYarQekSw0qokId5tgqU7/view?usp=sharing
https://app.diagrams.net/#G1ni_RgGjN7F9XYarQekSw0qokId5tgqU7#%7B%22pageId%22%3A%22PFwpzzzpqi9npyRdYy_4%22%7D

1.Design Principles Applied

Based on the system architecture diagrams and the targeted UI wireframes, the translation of EchoSphere's user stories into a concrete design demonstrates a strict adherence to core software engineering principles. Here is how the design practically implements these concepts:

- **1.1 Modularity:** The UI wireframes (ConceptExplainer, SmartSummarizer, KnowledgeGraph, UploadModal, SavedPrompts) represent distinct, self-contained feature workspaces. This translates directly to a component-driven React frontend. Because the UI is strictly modular, the backend mirrors this with dedicated controllers and services. If a new feature is introduced in the future, it can be built as an entirely new module without risking regressions in the existing codebase.
- **1.2 High Cohesion:** Each screen and backend module possesses a single, well-defined responsibility.
    - *UI Cohesion:* The Dashboard serves purely as an overview, while the SavedPrompts view is strictly for prompt management. The UI avoids forcing the user to manage settings on the same screen where they are trying to read an AI summary.
    - *Code Cohesion:* Backend services like backend/rag.js are highly cohesive. That specific service is responsible *only* for the Retrieval-Augmented Generation logic (embedding, vector search, LLM querying) and is not tangled with external concerns like PDF parsing or user session management.

Example:
javascript
```
// Backend: High Cohesion in documentController.js
export const uploadDocument = async (req, res) => {
    const file = req.file;
    const extractedText = await textExtractionService.extract(file); // Delegated
    const documentRecord = await DocumentModel.create({ name: file.name, content: extractedText });

    res.status(201).json(documentRecord);
};
```

- **1.3 Loose Coupling:** The architecture dictates a cleanly separated system where the frontend, backend API, LLM, and database operate independently.
    - *Client-Server Decoupling:* The React frontend and Node/Express backend communicate purely via REST APIs. The frontend is agnostic to how a PDF is chunked, and the backend is agnostic to how the knowledge graph is visually rendered.
    - *LLM Decoupling:* The application abstracts the AI provider. Because users inject their own API keys, the core system logic is loosely coupled to the specific LLM. Switching from Gemini to local Ollama models leaves the core document ingestion and database structures entirely unaffected.
- **1.4 Abstraction:** EchoSphere abstracts the immense mathematical complexity of Vector Embeddings and RAG pipelines to create distraction-free, intuitive interfaces for non-technical users. When interacting with the ConceptExplainer, users simply type a question and select a "depth" slider. The system completely abstracts the heavy lifting: chunking the text, generating vector embeddings in LanceDB, performing semantic similarity searches, and orchestrating the final LLM prompt injection.

- **Example:** The frontend simply initiates a declarative HTTP request:
- javascript

```javascript
// Frontend: Declarative, abstracted API call
const generateGraph = async (documentId) => {
    const response = await fetch(`/api/generate-graph/${documentId}`);
    return response.json();
};
```

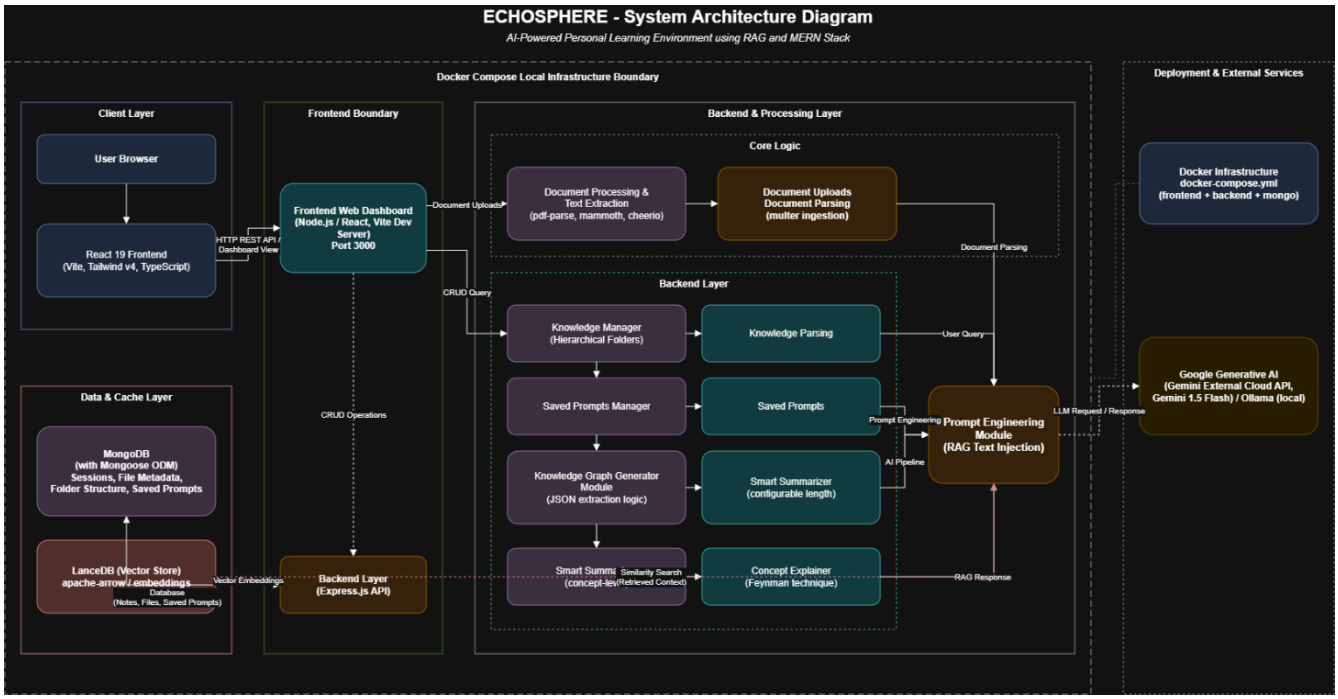
- **1.5 UI/UX Code Structuring:** The wireframes rely on a centralized, persistent navigation sidebar driving a main content area. This allows for a clean Router setup on the frontend. State management is efficiently localized; for instance, the KnowledgeGraph component manages its own react-force-graph physics state, while global state (such as the active Light/Dark theme or the currently loaded Document ID) is lifted to a higher-level context, keeping data flow predictable and maintainable.

2.High-Level Architecture

Chosen Architecture Style: RESTful Web API Architecture (Stateless Client-Server with Advanced RAG Pipeline)

- **Dedicated Vector Storage (Abstraction & Modularity):** The most critical update is the addition of LanceDB as a dedicated vector store in the Data & Cache Layer, utilizing apache-arrow for embeddings. Previously, data persistence relied solely on MongoDB. This architectural split ensures high cohesion: MongoDB exclusively handles relational metadata, session data, and folder structures, while LanceDB is optimized purely for high-dimensional vector search.

- **Precision Context Retrieval:** The data flow now features a explicit "Similarity Search (Retrieved Context)" pathway mapping directly from LanceDB to the Prompt Engineering Module. This justifies the RAG architecture by ensuring the AI pipeline only receives highly relevant, mathematically matched text chunks before executing the final LLM request.
- **Expanded Ingestion Engine:** The core Document Processing logic has been expanded to include cheerio alongside pdf-parse and mammoth. This decision increases the system's utility, allowing it to parse HTML and web-based study materials alongside standard PDFs and Word documents.
- **LLM Provider Agnosticism (Resilience):** To address the DA1 constraint regarding reliance on third-party APIs, the Deployment layer now supports local LLM execution via Ollama as an alternative to the Google Generative AI (Gemini 1.5 Flash) cloud service. This provides a highly secure, offline fallback for processing sensitive personal documents.
- **Containerized Independence:** The entire architecture, including the new LanceDB integration, remains strictly encapsulated within the Docker Compose Local Infrastructure Boundary. This ensures that adding complex vector search mechanics does not complicate the local development setup or deployment process.



Link : https://drive.google.com/file/d/1ni_RgGjN7F9XYarQekSw0qokId5tgqU7/view?usp=sharing

3. User Interface Design & Usability Justification

The user interface for EchoSphere was meticulously designed in Figma to minimize cognitive load and create a distraction-free study environment. By adhering to established UX heuristics, the application ensures that students and researchers can focus on learning rather than navigating a complex system.

Here is how the UI was made highly user-friendly across the 6 core screens:

- **Persistent & Predictable Navigation (All Screens):** The interface utilizes a persistent, universally recognized left-hand sidebar. This guarantees that users always know exactly where they are within the application and can context-switch between the Knowledge Base, Smart Summarizer, and Knowledge Graph with a single click, eliminating deep navigation trees.
- **Reduced Visual Fatigue (Global Theme):** The application employs an "Enterprise Dark Mode" globally. Given that target users (students and researchers) spend hours reading dense materials, the dark theme drastically reduces eye strain and improves readability during extended study sessions.
- **Contextual Side-by-Side Workspaces (Smart Summarizer):** Instead of forcing users to switch tabs between their source document and the AI output, the Smart Summarizer uses a split-screen design. The original PDF is rendered on the left, while the AI-generated summary is rendered simultaneously on the right, allowing for immediate cross-referencing and fact-checking.
- **Intuitive Mental Models (Upload Modal & Dashboard):** The Knowledge Base dashboard mimics standard file explorers (using hierarchical "Collections" and grid-based recent documents). The Upload Modal utilizes a familiar drag-and-drop hit area, lowering the learning curve for non-technical users.
- **Progressive Disclosure of Complexity (Concept Explainer):** The Concept Explainer does not overwhelm the user with complex prompt engineering options. Instead, it uses a simple visual slider (ranging from "Layperson" to "Expert"). This translates a complex backend prompt injection process into a highly intuitive, physical-feeling UI control.
- **Decluttered Visualization (Knowledge Graph):** Data visualizations can easily overwhelm the user. The Knowledge Graph screen maximizes screen real estate by making the 2D physics graph full-screen, while tucking complex rendering settings (Auto-expand, Layout styles, Labels) into a collapsible side panel.
- **Workflow Acceleration (Saved Prompts):** To prevent users from typing repetitive instructions, the Saved Prompts screen organizes custom queries by category tags (e.g., "Exams", "Research") and provides one-click "Use prompt" buttons. This respects the user's time and accelerates their study workflow.

4. Design Decisions & Rationale

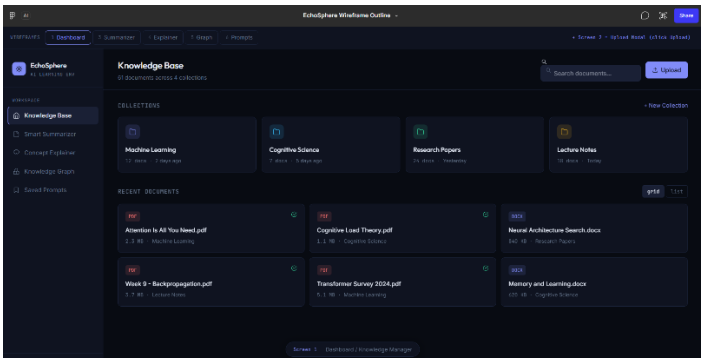
During the architectural design of EchoSphere, several pivotal technical choices were made to optimize development speed, system maintainability, and resource allocation. The following outlines the primary design decisions and their underlying justifications:

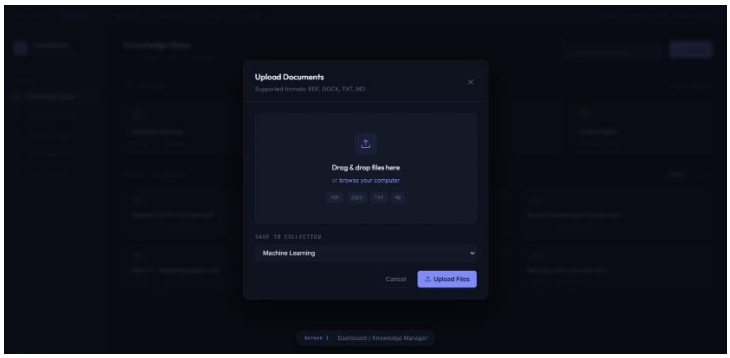
- **Decision: Client-Side API Key Management via localStorage** *Why:* To mitigate the substantial architectural overhead of implementing a centralized user authentication, role-based access control (RBAC), and subscription billing system. By offloading the Google Gemini 1.5 Flash API key management securely to the client's local browser storage, the application maintains a strict "Bring Your Own Key" (BYOK) model. This eliminates backend liability for API quota exhaustion,

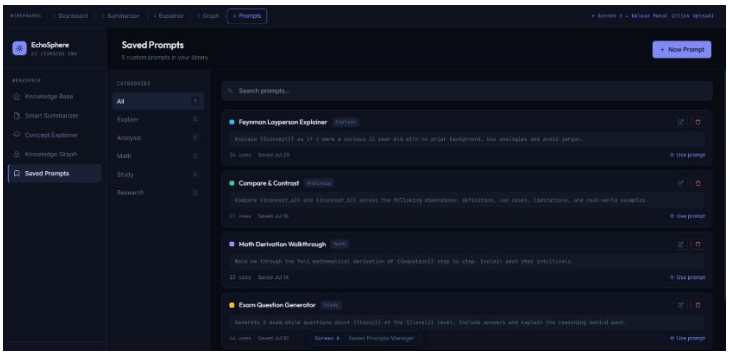
significantly reduces database complexity, and allows the development focus to remain exclusively on core RAG capabilities rather than user identity management.

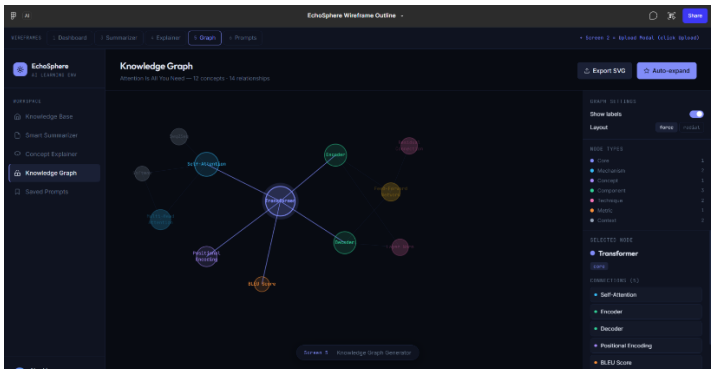
- **Decision: Utilizing react-force-graph for Knowledge Graph Visualization** *Why:* To adhere to the principle of not "reinventing the wheel" when handling complex, physics-based rendering. Building a custom WebGL, SVG, or Canvas-based node rendering engine from scratch would introduce unnecessary risk and require significant development time to handle edge cases like collision detection, zooming, and panning. Integrating the established react-force-graph library provided immediate, robust support for D3-based force-directed layouts, ensuring high-performance graph visualizations with minimal technical debt.
- **Decision: Decoupling Frontend and Backend via Stateless REST API and Dockerization** *Why:* To enforce strict separation of concerns and maximize horizontal scalability. By enforcing a stateless HTTP API contract, the React 19 UI and Node.js/Express backend operate in total isolation. Containerizing both environments using Docker guarantees deterministic execution across all deployment environments (development, staging, and production). This decoupled architecture prevents vendor lock-in, ensures that memory leaks in the backend do not crash the UI, and allows either component to be scaled or refactored independently.
- **Decision: Centralizing Document Extraction on the Backend (pdf-parse / mammoth)** *Why:* To ensure deterministic parsing behavior, protect client-side performance, and maintain a centralized ingestion pipeline. While client-side parsing libraries exist, executing heavy I/O operations and text extraction algorithms (for PDFs and DOCX files) within the browser thread risks severely degrading the user experience, especially on low-powered devices. By utilizing dedicated backend libraries, the architecture guarantees that document processing, text chunking, and subsequent vectorization are handled synchronously within a highly controlled server environment before being persisted to MongoDB.

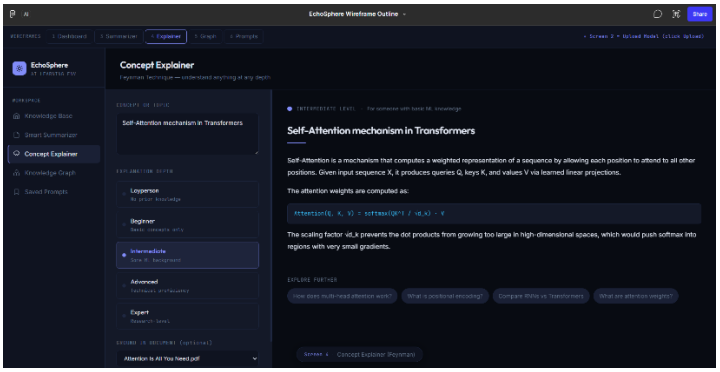
5. Figma Diagrams: (Already attached in DA1)

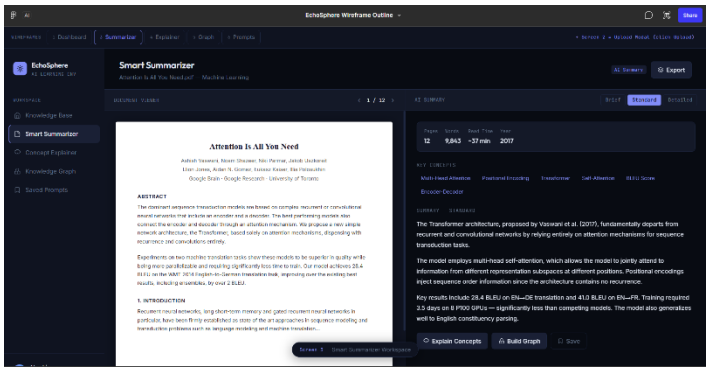








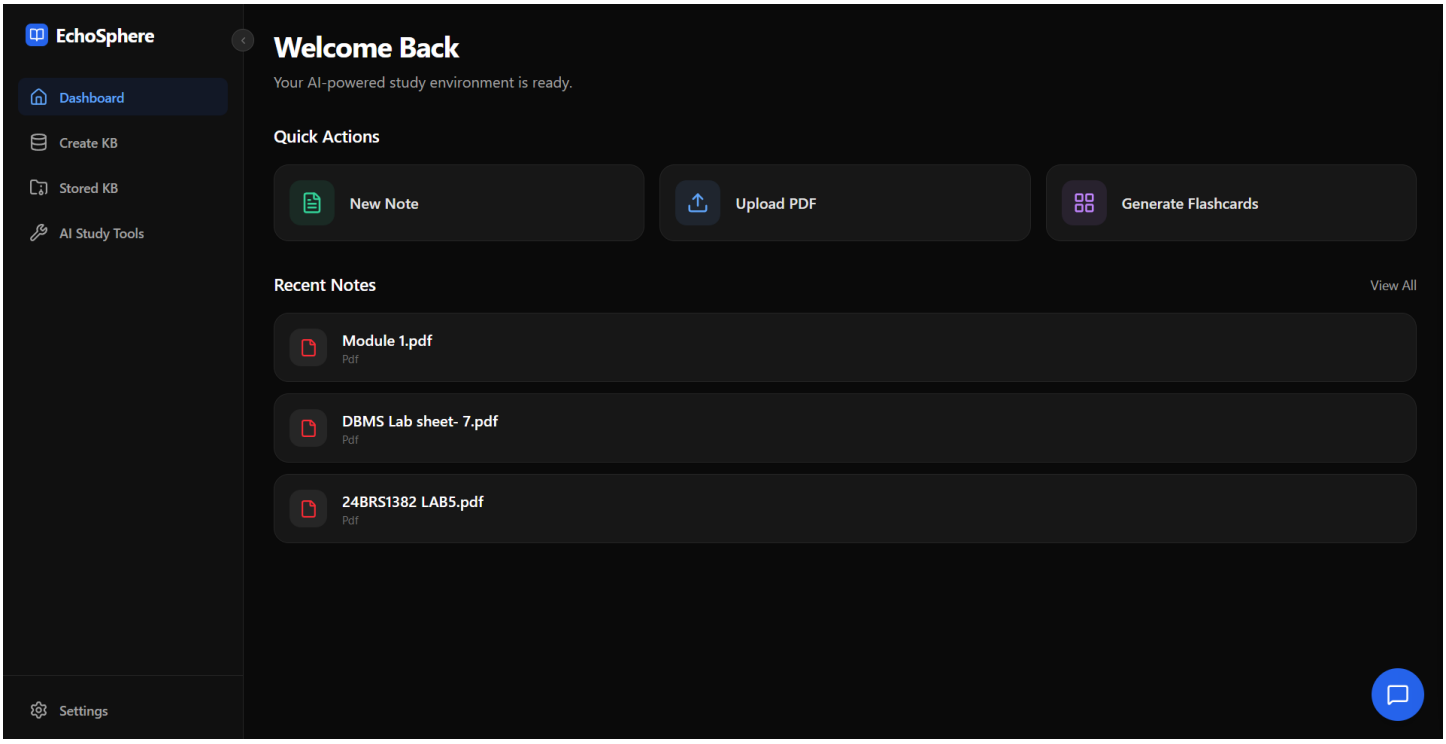




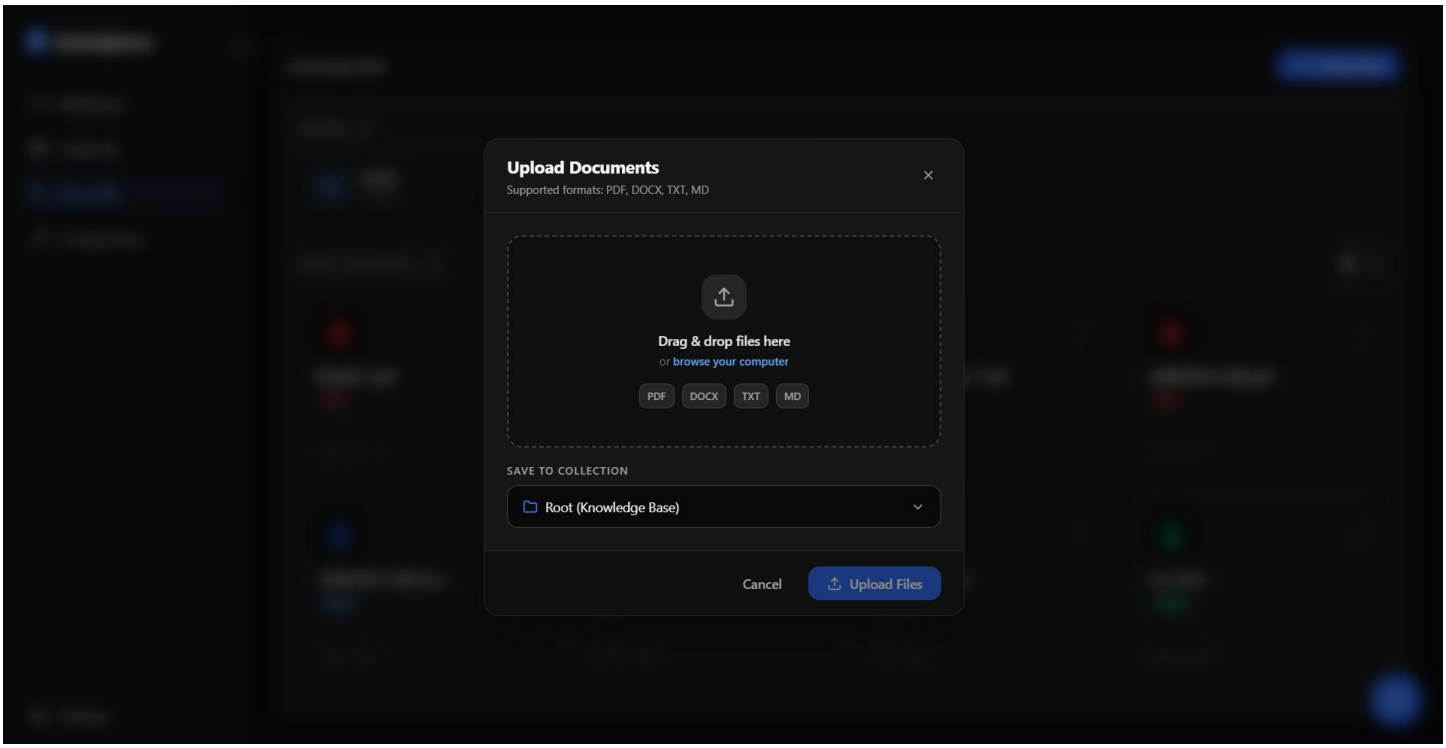
Figma Link : https://www.figma.com/make/o0oFJ9qgmdbQpxuekEYeyl/EchoSphere-Wireframe-Outline?t=tsLKVe3ktEjJpQcJ-20&fullscreen=1

Actual project Screenshots:

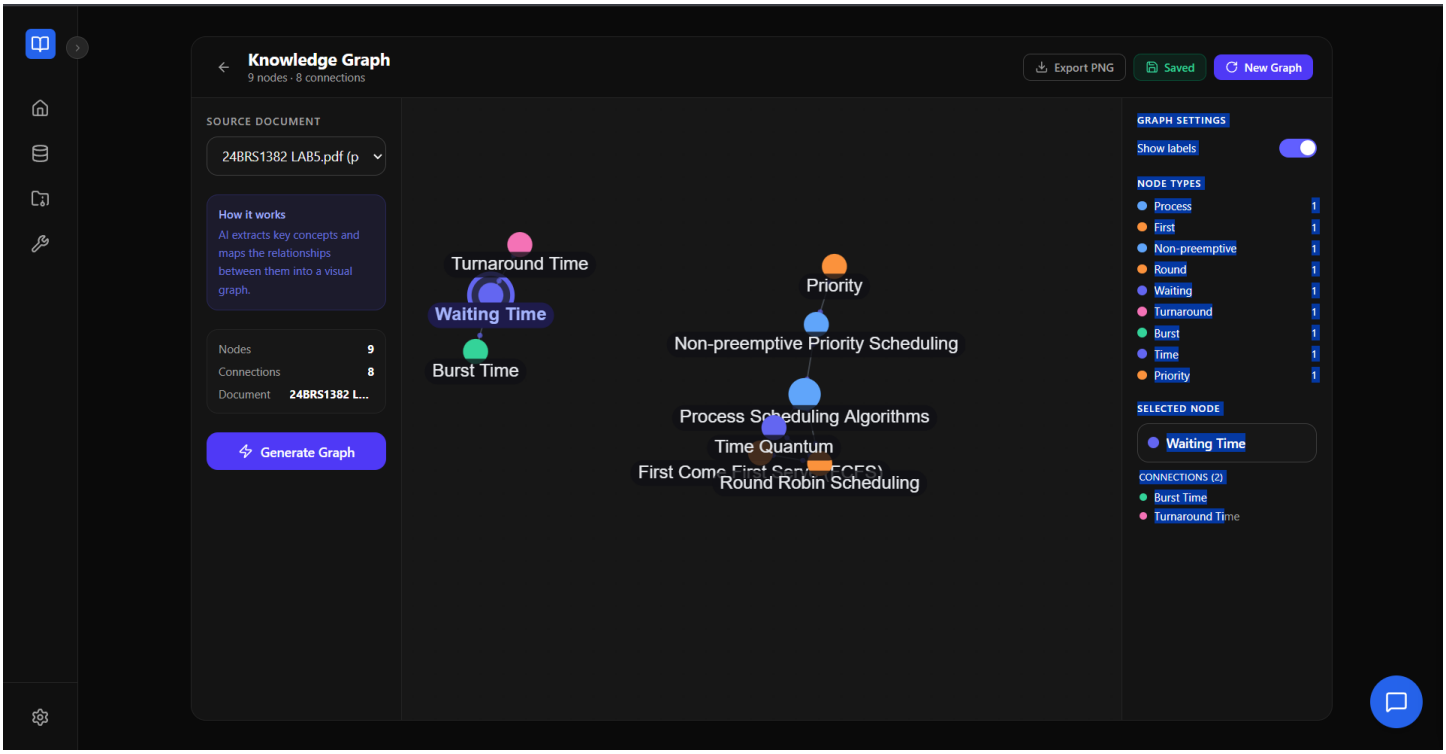
Screenshot 1: Dashboard & Workspace Entry (Displaying "Welcome Back", Quick Actions, and Recent Notes like "DBMS Lab sheet- 7.pdf").



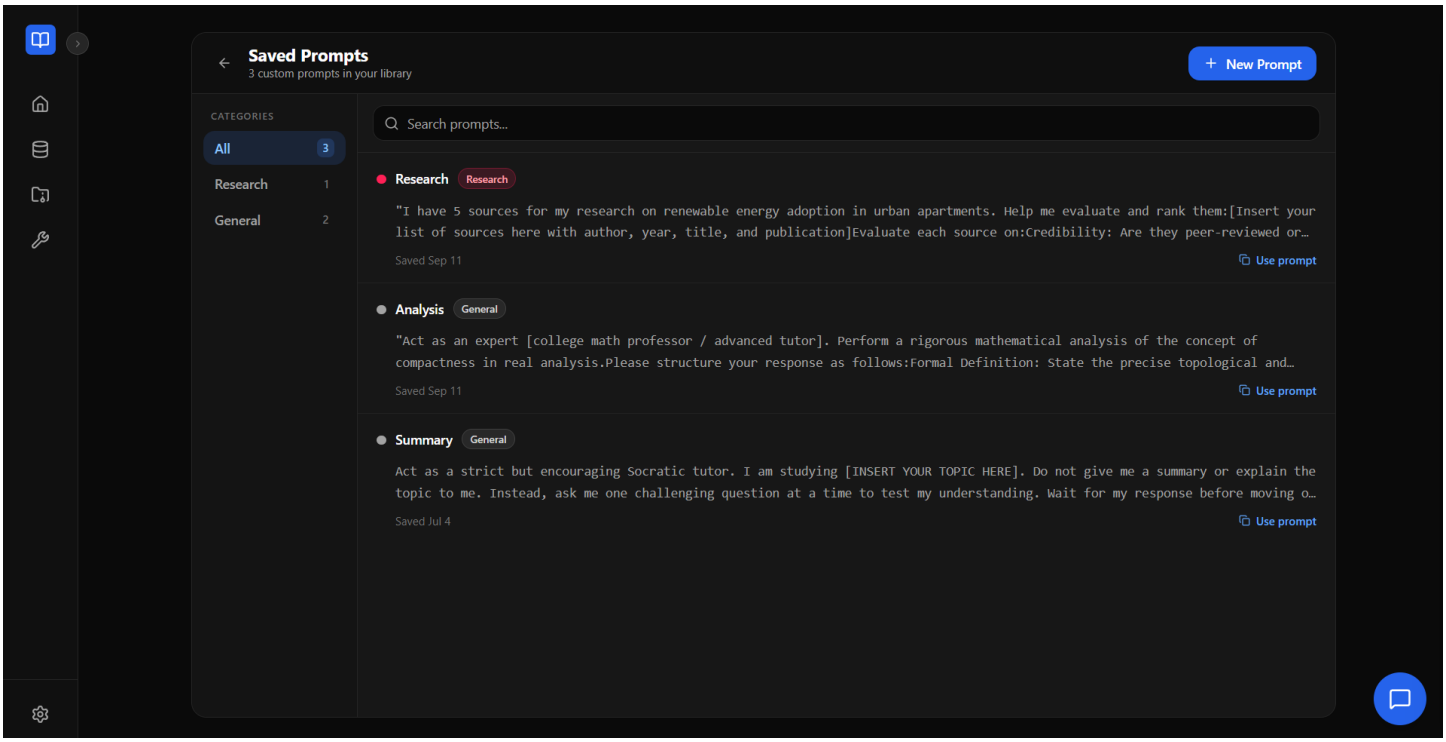
Screenshot 2: Document Upload Modal (Displaying drag-and-drop functionality for PDF, DOCX, TXT, MD files saving to "Root (Knowledge Base)").



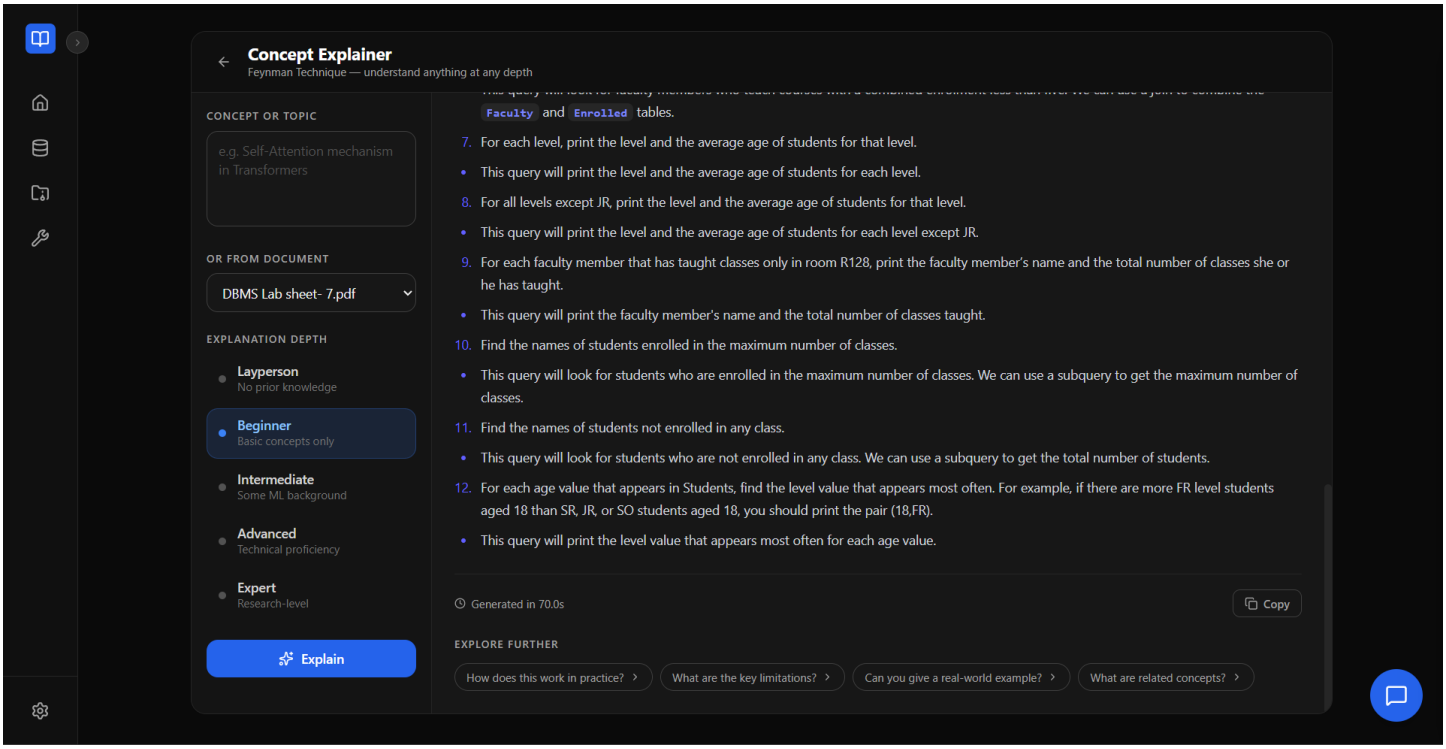
Screenshot 3: Interactive Knowledge Graph (Displaying the 2D physics graph extracting nodes like "Turnaround Time" and "Burst Time" from the "24BRS1382 LAB5.pdf" document).



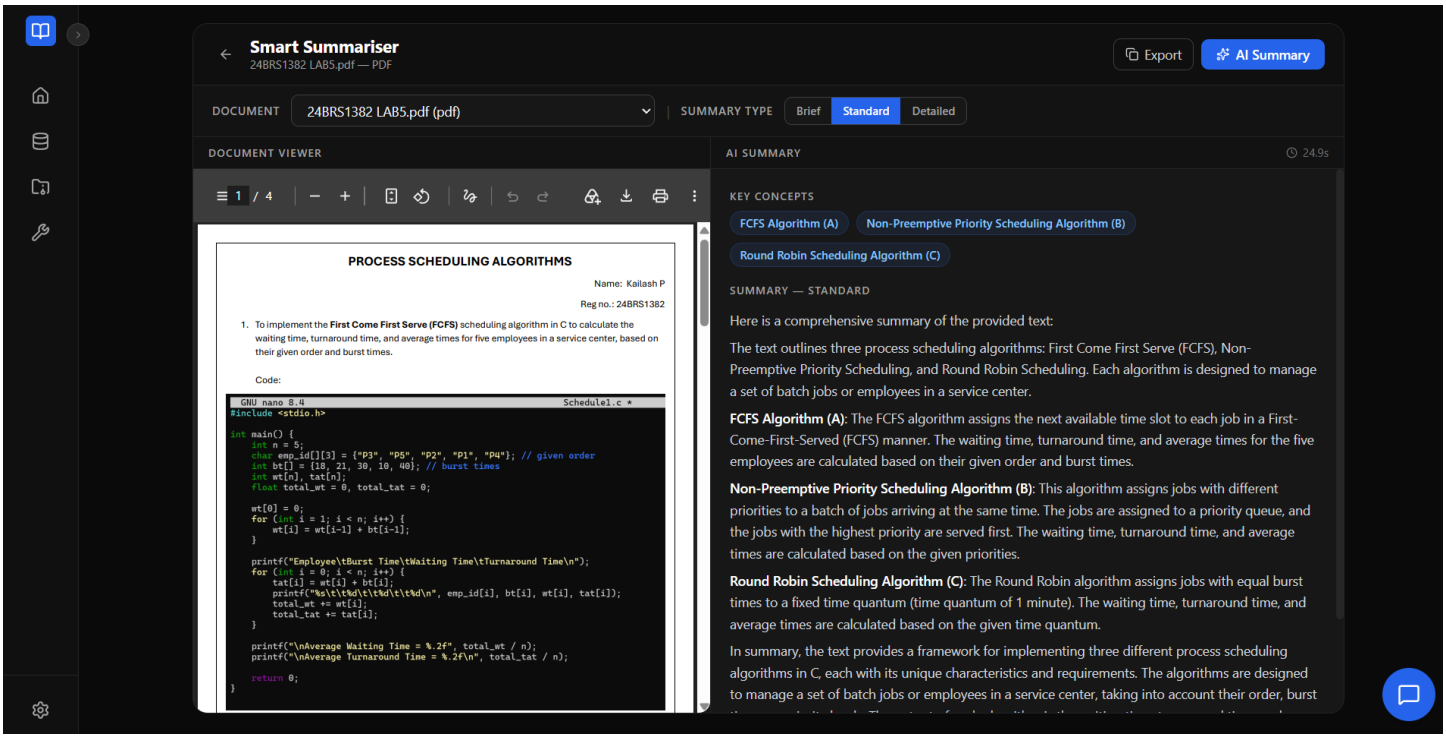
Screenshot 4: Saved Prompts Manager (Displaying categorized custom AI prompts for Research, Analysis, and Summary).



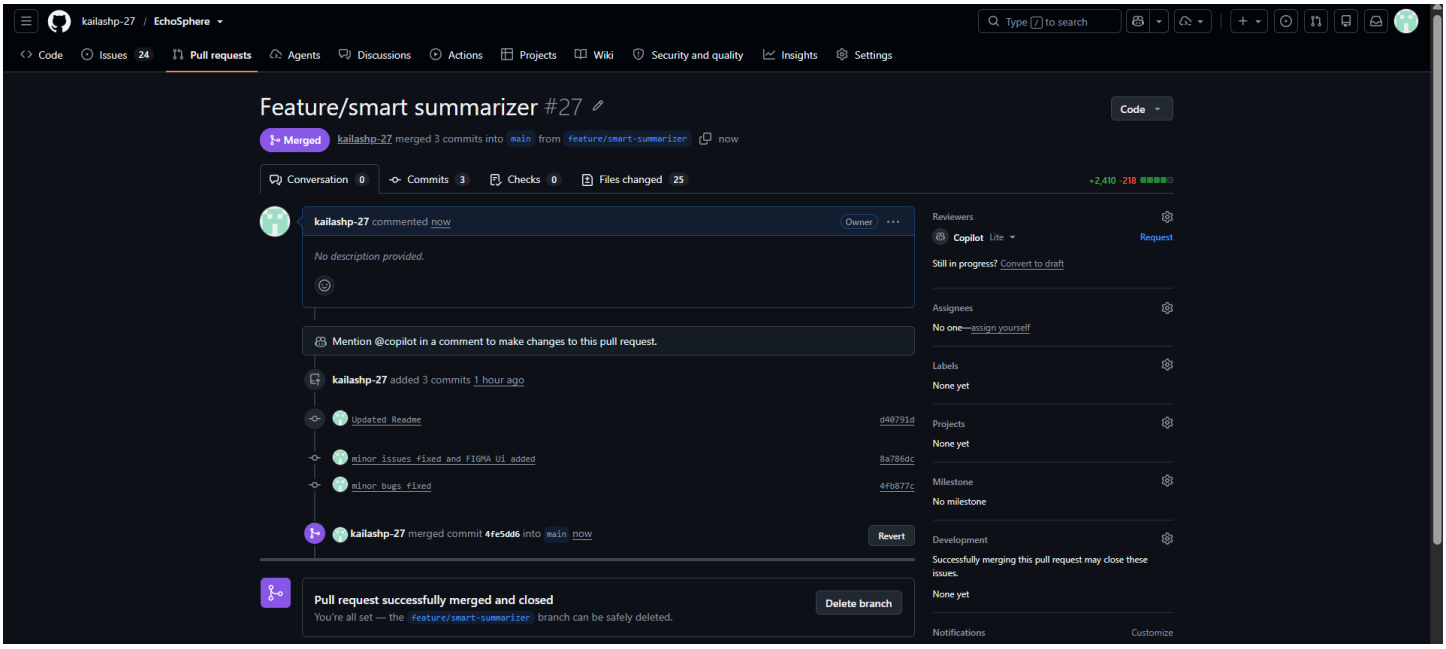
Screenshot 5: Concept Explainer (Feynman Technique) (Displaying the depth selection slider set to "Beginner" and explaining database concepts from the DBMS lab sheet).



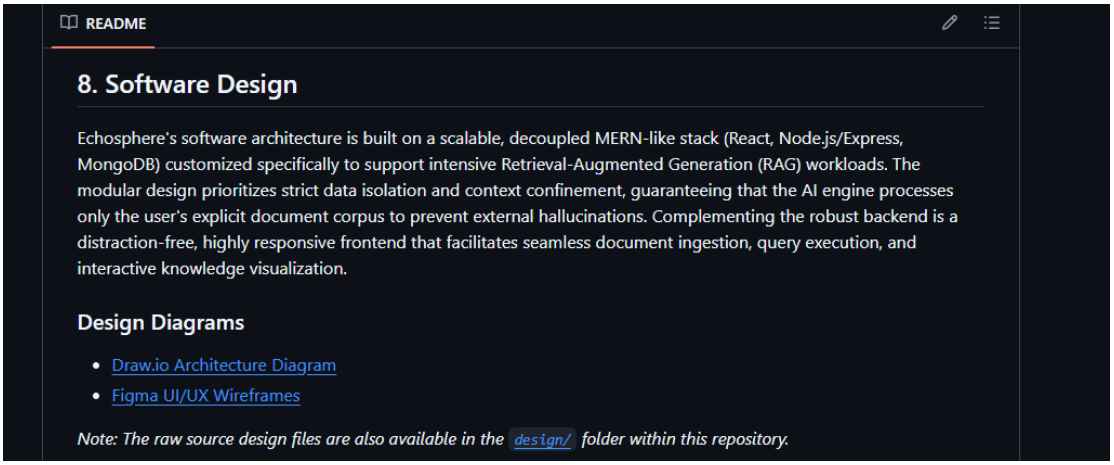
Screenshot 6: Smart Summarizer (Displaying the split-screen view with the source C programming PDF on the left and the generated AI summary of scheduling algorithms on the right).



GitHub Flow Validation - Successful Pull Request and Merge of feature/smart-summarizer into main



README.md Update - Software Design Section and Diagram Link



6. System Maintainability & Future Extensibility

A core objective in designing EchoSphere was to ensure the system is not only functional for its initial release but also highly maintainable and adaptable to future technological shifts. The architecture anticipates growth in several key areas:

6.1 Designing for Maintainability

- **Predictable Debugging via MVC:** By enforcing a strict Model-View-Controller pattern on the backend, bugs are isolated. If an error occurs during text extraction, developers know exactly to look in the ingestion service, rather than hunting through API routing logic.
- **API Versioning and Contracts:** The stateless REST API acts as a strict contract between the frontend and backend. Future UI overhauls (e.g., migrating from Tailwind v4 to a different CSS framework, or upgrading React versions) will not require any changes to the backend infrastructure, making maintenance highly predictable.
- **Environment Consistency:** Containerizing the application via Docker Compose ensures that "it works on my machine" issues are eliminated. Any future maintainer can spin up the exact same development environment—including the LanceDB vector store and MongoDB instance—with a single command (docker-compose up).

6.2 Anticipating Future Changes (Extensibility)

- **LLM Provider Agnosticism:** The AI landscape evolves rapidly. Because the Prompt Engineering module is loosely coupled to the LLM API, swapping out Gemini 1.5 Flash for a future model (like GPT-5, Claude 3.5, or expanded local Ollama models) simply requires writing a new API wrapper service, leaving the core RAG logic and database intact.
- **Extensible Document Ingestion:** The current ingestion pipeline supports PDFs, DOCX, and HTML. Because document parsing is modularized, future support for new formats—such as PowerPoint (.pptx) or audio transcripts via Whisper—can be achieved by simply slotting a new parser library into the existing ingestion controller.